

Cloud infrastructure

[Menu](#)[On this page](#)

Séance 7

Remarque

L'image à utiliser est [docker.io/5clo1r/k8s-labo](https://esibru.github.io/5clo1r/k8s-labo) avec le tag **0.1.0**

Les probes liveness et readiness doivent toujours être définies. Elles sont de type HTTP et sont disponibles sur les path `/liveness` et `/readiness`.

Stockage

Les *Volumes* permettent de stocker des données de manière persistante, partagée entre différents *Pod*. Ils répondent à la limite liée au cycle de vie d'un *Pod* : **le système de fichier d'un *Pod* est éphémère.**

On peut utiliser les volumes pour préserver les données quand un *Pod* redémarre, accéder à un système de stockage externe ou injecter des éléments de configuration via des *ConfigMaps* ou *Secrets*.

Différent types de volumes existent :

- *emptyDir* : un volume vide est créé à la création du *Pod* et disparaît à la suppression du *Pod*;
- *ConfigMaps* et *Secret* : injectent des éléments de configuration dans un *Pod*;
- *PersistentVolume* et *PersistentVolumeClaims* : permettent un stockage persistant indépendant du cycle de vie du *Pod*. On peut avoir un provisioning dynamique à l'aide de *StorageClass*.

Accéder au filesystem d'un *Pod*

Pour accéder au filesystem d'un *Pod*, il est possible d'exécuter des commandes dans le *Pod* à l'aide de la commande `kubectl exec -it <pod-name> -- /bin/sh`

Tâche 1

De manière déclarative, créez un *Deployment* avec un volume éphémère de type *emptyDir*.

Exigences
Le <i>Deployment</i> doit avoir un replica
Le <i>Volume</i> doit être appelé <i>tmp-data</i>
Le <i>Volume</i> doit être accessible via le path <code>/srv</code>

Exemple de *Pod* avec un volume de type *emptyDir*

yaml

```
---
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: my-container
      image: nginx
      volumeMounts:
        - name: shared
          mountPath: /usr/share/nginx/html
  volumes:
    - name: shared
      emptyDir: {}
```

Questions

Quel est le contenu du fichier yaml pour créer le *deployment* avec un volume de type *emptyDir*

Dans le *Pod* créez un fichier `/srv/fichier` , redémarrez le *Pod* et observez le contenu du répertoire `/srv` .

Tâche 2

De manière déclarative, créez un *Deployment* avec un volume persistant.

Exigences
Le <i>Deployment</i> doit avoir un replica
Le <i>Volume</i> doit être appelé <i>tmp-data</i>
Le <i>Volume</i> doit être accessible via le path <code>/srv</code>
Le <i>Volume</i> doit avoir une taille de 500Mo

Exemple d'un *PersistentVolumeClaims* utilisant une *StorageClass* nommée *local-path*

yaml

```
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: local-path
  resources:
    requests:
      storage: 1Gi
```

Remarque

Le `storageClassName` de type `local-path` est propre à notre environnement — *k3s* en l'occurrence. Les *StorageClass* doivent donc être définis au préalable. Avec *k3s* et *Minikube*, un *StorageClass* de type *local-path* est prédéfini automatiquement.

Exemple de *Pod* utilisant un volume créé via un *PersistentVolumeClaims*

yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pvc
spec:
  containers:
```

```
- name: app
  image: nginx
  volumeMounts:
    - name: data
      mountPath: /data
volumes:
- name: data
  persistentVolumeClaim:
    claimName: my-pvc
```

Questions

Quel est le contenu du fichier yaml pour créer le *deployment* avec un volume via un *PersistentVolumeClaim* ?

Dans le *Pod* créez un fichier `/srv/fichier` , redémarrez le *Pod* et observez le contenu du répertoire `/srv` .

Tâche 3

De manière déclarative, créez un *StatefulSet* avec un volume *via* un *VolumeClaimTemplates*.

Exigences
Le <i>StatefulSet</i> doit avoir 3 replicas
Le <i>Volume</i> doit être appelé <i>tmp-data</i>
Le <i>Volume</i> doit être accessible via le path <code>/srv</code>
Le <i>Volume</i> doit avoir une taille de 500Mo
La <i>StorageClass</i> à utiliser est <i>local-path</i>

Exemple partiel d'un *StatefulSet* avec un *volumeClaimTemplates*

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
```

yaml

```
  name: web
spec:
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: "my-storage-class"
      resources:
        requests:
          storage: 1Gi
```

Questions

Quel est le contenu du fichier yaml pour créer le *StatefulSet* ?

Comment sont nommés les différents *PersistentVolumeClaims* ? Quelle est la relation entre le nom des *Pod* ?

Dans chaque replica, créez un fichier `/srv/fichier-{idReplica}` , redémarrez les *Pod* et observez et comparez le contenu du répertoire `/srv` pour chaque *Pod*.

Mise à jour le: lundi 3 novembre 2025 à 11:17

Previous page
[Séance 6](#)

Next page
[Séance 8](#)